



CENTRO EDUCATIVO DEPARTAMENTAL MUNICIPAL

MÍGUELA RODRÍGUEZ ARAUJO — Luque, Paraguay

Java: ciclos

while, do-while y for — repetir instrucciones con criterio

Laboratorio · Gabinete de Informática — 3º BATIN — Etapa 2

Continuación de "Java desde cero" y "Java: condicionales"

"Saber, hacer y transformar"

¿Qué vamos a lograr hoy?

- 1 Entender por qué necesitamos repetir instrucciones sin copiar y pegar código
- 2 Usar while para repetir mientras una condición sea verdadera
- 3 Usar do-while cuando necesitamos que el bloque se ejecute al menos una vez
- 4 Usar for cuando ya sabemos cuántas veces queremos repetir
- 5 Combinar ciclos con condicionales, y anidar un ciclo dentro de otro

¿Por qué necesitamos repetir instrucciones?

- Bit y Techy ya tienen su sistema leyendo y decidiendo con `if`. Pero si quieren registrar 20 ventas, ¿van a escribir 20 veces el mismo bloque de código?
- Un ciclo (o bucle) le dice al programa: repetí este bloque de instrucciones mientras se cumpla una condición, o una cantidad de veces determinada.
- Todo ciclo necesita, en algún momento, una condición de corte — la misma idea de comparación que ya usamos con los condicionales la semana pasada.

¿Cuál uso en cada caso?

while

Repito MIENTRAS se cumpla una condición. No sé de antemano cuántas vueltas van a ser.

Ejemplo:

Validar una entrada hasta que sea correcta.

do-while

Igual que while, pero el bloque se ejecuta SIEMPRE al menos una vez, antes de preguntar.

Ejemplo:

Mostrar un menú y repetir mientras elijan seguir.

for

Repito una cantidad de veces que YA CONOZCO, con un contador incorporado en una sola línea.

Ejemplo:

Recorrer del 1 al 10, o leer N datos.

01

while

Repetir mientras una condición sea verdadera. La condición se revisa antes de cada vuelta.

Sintaxis del while

- **Estructura:** `while (condición) { instrucciones }` — mientras la condición sea `true`, el bloque se ejecuta una y otra vez.
- **La condición se revisa primero:** si es `false` desde el principio, el bloque no se ejecuta ni una sola vez.
- **Cuidado:** algo dentro del bloque tiene que cambiar la condición eventualmente — si no, el ciclo nunca termina (ciclo infinito).

Ejemplo: contar del 1 al 5

```
public class ContarDel1Al5 {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

Ejemplo: validar una entrada

- El while también sirve para repetir mientras un dato NO cumpla lo que necesitamos.

```
import java.util.Scanner;

public class ValidarPositivo {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int numero = entrada.nextInt();

        while (numero < 0) {
            System.out.println("Número negativo, probá de nuevo:");
            numero = entrada.nextInt();
        }
        System.out.println("Ingresaste: " + numero);
    }
}
```


02

do-while

El bloque se ejecuta PRIMERO, y la condición se revisa DESPUÉS — por eso siempre corre al menos una vez.

Sintaxis del do-while

- **Estructura:** `do { instrucciones } while (condición);` — fíjate que acá el `while` va al FINAL, con punto y coma.
- **Diferencia clave con `while`:** el bloque se ejecuta una vez sí o sí, incluso si la condición ya es falsa desde el arranque.
- **Cuándo usarlo:** cuando la acción tiene sentido hacerla al menos una vez antes de preguntar — por ejemplo, pedir una clave.

Ejemplo: intentos de acceso

- El programa pide la clave las veces que hagan falta, hasta que coincida.

```
import java.util.Scanner;

public class IntentosAcceso {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        String claveCorrecta = "feria2026";
        String intento;
        int intentos = 0;

        do {
            System.out.println("Ingresa la clave:");
            intento = entrada.nextLine();
            intentos++;
        } while (intento != claveCorrecta);

        System.out.println("Acceso en el intento " + intentos);
    }
}
```

03

for

El ciclo con contador incorporado: inicialización, condición y actualización, todo en una sola línea.

Sintaxis del for

- **Estructura:** `for (inicialización; condición; actualización) { instrucciones }`
- **Inicialización:** se ejecuta una sola vez, al principio (ej.: `int i = 1;`)
- **Condición:** se revisa antes de cada vuelta, igual que en el `while`
- **Actualización:** se ejecuta al final de cada vuelta, después del bloque (ej.: `i++`)

FOR

Ejemplo: tabla de multiplicar

```
public class TablaMultiplicarFor {  
    public static void main(String[] args) {  
        int numero = 6;  
        for (int i = 1; i <= 10; i++) {  
            System.out.println(numero + " x " + i + " = " + (numero * i));  
        }  
    }  
}
```

Ejemplo: for + Scanner + acumulador

- Un patrón muy común: leer N datos dentro de un for, acumulando un resultado.

```
import java.util.Scanner;

public class SumarVentasConFor {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int cantidadVentas = entrada.nextInt();

        double total = 0;
        for (int i = 1; i <= cantidadVentas; i++) {
            double monto = entrada.nextDouble();
            total += monto;
        }
        System.out.println("Total: Gs. " + total);
    }
}
```

04

Combinando ciclos

*Un for dentro de otro for, y ciclos junto con if —
para resolver problemas más completos.*

Ejemplo: tabla de multiplicar completa (for dentro de for)

- Por cada vuelta del for externo, el for interno completa TODAS sus vueltas.

```
public class TablaMultiplicarCompleta {  
    public static void main(String[] args) {  
        for (int numero = 1; numero <= 3; numero++) {  
            for (int i = 1; i <= 5; i++) {  
                System.out.println(numero + " x " + i + " = " + (numero * i));  
            }  
        }  
    }  
}
```

Ejemplo: sistema de ventas del día

- Un for que lee datos, acumula un total, y clasifica cada dato con un if — el patrón más completo de la clase.

```
import java.util.Scanner;

public class SistemaVentasDelDia {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int cantidadVentas = entrada.nextInt();

        double total = 0;
        int ventasGrandes = 0;
        for (int i = 1; i <= cantidadVentas; i++) {
            double monto = entrada.nextDouble();
            total += monto;
            if (monto >= 10000) {
                ventasGrandes++;
            }
        }
    }
}
```

Errores comunes con ciclos

- **Ciclo infinito:** olvidarse de cambiar la variable de control dentro del ciclo (por ejemplo, olvidar el `i++`). La práctica interactiva detiene el programa sola después de 20.000 vueltas y te avisa.
- **Confundir while y do-while:** usar do-while cuando en realidad la condición debería revisarse ANTES de la primera ejecución.
- **Inicializar mal el acumulador:** para sumar, el acumulador empieza en 0. Para multiplicar (como en un factorial), tiene que empezar en 1 — si empieza en 0, el resultado siempre va a ser 0.
- **Dividir dentro del ciclo por error:** en un promedio, la división se hace UNA vez al final, no en cada vuelta.

20 ejercicios en una página nueva

La práctica de ciclos vive en una página separada: "Java: ciclos". Cubre, de menor a mayor dificultad:

while

Ejercicios 1–5

Contar, sumar, validar entrada, vender hasta agotar stock, tabla de multiplicar

do-while

Ejercicios 6–9

Ejecutar al menos una vez, cuenta regresiva, intentos de acceso, suma con centinela

for

Ejercicios 10–15

Tabla de multiplicar, suma, contar pares, factorial, repetir mensaje, sumar ventas

Combinados y anidados

Ejercicios 16–20

For anidado, pares e impares, buscar múltiplo, promedio, sistema de ventas integrador

Lo que viene

- Hoy sumamos los tres ciclos (while, do-while, for), y cómo combinarlos con condicionales y anidarlos entre sí.
- Con esto ya tenemos las tres herramientas base de la lógica de programación: variables, condicionales y ciclos.
- La próxima clase de Laboratorio seguimos con arreglos — y ahí retomamos directamente la conexión con las clases del sistema propio de cada equipo, usando ciclos para recorrerlos.